

Ejercicio 1

Debido a un error en la actualización de sus sistemas, el banco AyED perdió la información del estado de todas sus cuentas. Afortunadamente logran recuperar un backup del día anterior y utilizando las transacciones registradas en las últimas 24hrs podrán reconstruir los saldos. Hay poco tiempo que perder, el sistema bancario debe volver a operar lo antes posible. Las transacciones se encuentran en consultas, una consulta cuenta con un valor y un rango de cuentas consecutivas a las que hay que aplicar este cambio, por ejemplo la consulta ($333.688 = 120$) implica sumar \$120 a todas las cuentas entre la número 333 y la número 688 (inclusive). Entonces, la recuperación de los datos consiste en aplicar todas las consultas sobre el estado de las cuentas recuperadas en el backup del día anterior. El equipo de desarrollo se pone manos a la obra y llega a una solución rápidamente (Algoritmo **procesarMovimientos**). Toman cada consulta y recorren el rango de cuentas aplicando el valor correspondiente, como muestra el siguiente algoritmo.

```
Consultas.comenzar();
while(!consultas.fin()){
    Consultas = consultas.proximas();
    for (i= consulta.desde; i < consulta.hasta; i++){
        cuenta[i] = cuenta[i] + consulta.valor;
    }
}
```

Escriben la solución en pocos minutos y ponen en marcha el proceso de recuperación. Enseguida se dan cuenta que el proceso va a tardar muchas horas en finalizar, son muchas cuentas y muchos movimientos, la solución aunque simple es ineficiente. Luego de discutir varias ideas llegan a una solución (Algoritmo **procesarMovimientosOptimizado**) que logra procesar toda la información en pocos segundos. Ambos algoritmos se encuentran en el archivo Ejercicio 1 - rsq-tn_ayed.zip del material adicional.

a.- Para que usted pueda experimentar el tiempo que demora cada uno de los dos algoritmos en forma empírica, usted debe ejecutar cada uno de ellos, con distintas cantidades de elementos y completar la tabla. Luego haga la gráfica para comparar los tiempos de ambos algoritmos. Tenga en cuenta que el algoritmo posee dos constantes **CANTIDAD_CUENTAS** y **CANTIDAD_CONSULTAS**,

sin embargo, por simplicidad, ambas toman el mismo valor. Sólo necesita modificar CANTIDAD_CUENTAS.

N° Cuentas (y consultas)	procesarMovimientos	procesarMovimientosOptimizado
1.000	0,039	0,002
10.000	0,545	0,009
25.000	4,044	0,021
50.000	15,552	0,024
100.000	60,337	0,034

b.- ¿Por qué procesarMovimientos es tan ineficiente? Tenga en cuenta que pueden existir millones de movimientos diarios que abarquen gran parte de las cuentas bancarias.

Es ineficiente porque por cada consulta recorre uno por uno todos los índices de su rango para sumar el valor. El costo de una consulta depende del tamaño del rango, no es constante. Con millones de movimientos que abarcan gran parte de las cuentas, el trabajo total es del orden de $n \times m$ (cuentas $\times$ consultas), un algoritmo cuadrático, $O(n^2)$.

c.- ¿En qué se diferencia procesarMovimientosOptimizado? Observe las operaciones que se realizan para cada consulta.

Por cada consulta hace solo 2 operaciones, sin importar el tamaño del rango: suma el valor en aux[desde] y resta ese mismo valor en aux[hasta+1]. Eso es $O(1)$ por consulta $\rightarrow O(m)$ en total. Después, un único recorrido final del arreglo, acumulando ($\text{aux}[i] += \text{aux}[i-1]$), propaga el efecto de cada consulta a lo largo de su rango y lo "corta" justo donde termina. Ese paso es $O(n)$.